

FULL DESK LEADS

Base Build Plan

A 14-day plan to stand up the walking skeleton: two signal sources moving end to end through every pipeline stage at basic depth, landing correctly in a tenant-isolated queue, with a thin interface to see it happen. Everything here is deliberately a base. Everything will be refined later.

Goal	Prove the full shape end to end at basic depth, not build any one stage to spec yet.
Architecture	One repository, one deployable backend. No service split between engine and platform logic.
Stack	Node.js + TypeScript, Fastify, BullMQ + Redis, Supabase (Postgres + Auth + RLS), Railway, React + Vite for the interface
Timeline	14 days total, including two margin days for testing and bug fixing

What Counts as the Base

Every pipeline stage exists and runs. None of them are built to full spec yet. A structural filter stands in for the full three-tier exclusion system. A single highest-confidence contact stands in for the weighted selection algorithm. A hardcoded placeholder stands in for generated intelligence. The point of these 14 days is a provably correct shape, not a finished pipeline.

WATCH ITEMS GOING IN

Greenhouse has no public directory of company board tokens, so the initial target list is built by hand against real companies. Basic contact selection in this phase picks the single highest-confidence contact rather than running the full weighted scoring algorithm, so primary-contact accuracy will improve materially in the next phase, not this one. Seamless.AI is the only enrichment source in this base phase, with no fallback provider, so real coverage on this specific target list is worth confirming early rather than assumed. The lead payload shape must be frozen on day 3 and treated as load-bearing from that point forward, since every later stage writes against it.

WEEK 1 — Foundation and Pipeline Skeleton

DAY 1 | Setup and Scaffolding

- Scaffold the repo: /api (Fastify), /workers (BullMQ), /shared (types, Supabase client, config), /migrations, /web (React + Vite frontend)
- Install core dependencies: fastify, bullmq, ioredis, @supabase/supabase-js, zod, pino, plus the frontend toolchain and Tailwind
- Create the Supabase project and Railway project (API, worker, and Redis services)
- Confirm API-level access on Seamless.AI, since paid plans are required for programmatic enrichment beyond the 50-credit free tier

EXIT CRITERIA

Repo is scaffolded, accounts exist, the bare Fastify server and the bare React app both boot locally, and the local machine reaches Redis.

DAY 2 | Core Data Schema and Row Level Security

- Write the initial migration covering every core object: tenant, seat, company, hiring_signal, contact, lead, lead_assignment, configuration, exclusion, score_record, interaction_event, raw_signal
- Add tenant_id to every tenant-scoped table and enable RLS on every table
- Write tenant_id-scoped policies for API-facing reads; service-role key bypasses RLS for workers; index every column referenced in a policy
- Seed two dummy tenants and two dummy seats, then confirm isolation by querying as each

EXIT CRITERIA

Schema is applied via migration, every core object exists with the correct relationships, and querying as either dummy tenant shows zero cross-tenant visibility.

DAY 3 | The Lead Contract and Queue Infrastructure

- Define the lead payload as a shared Zod schema and TypeScript type, covering the full field inventory even where a field stays null for now. This is the internal discipline boundary standing in for a service contract
- Define one BullMQ queue per pipeline stage: ingest, normalize, resolve-identity, filter, enrich, select-contact, score, emit
- Stand up worker skeletons (logging only) and wire up Bull Board for local visibility

EXIT CRITERIA

A manually enqueued test job flows through every worker skeleton and completes visibly in Bull Board. The shared lead type compiles and imports cleanly from both /workers and /api.

DAY 4 | Signal Source: Greenhouse

- Build a source_companies config table holding board tokens for real target companies
- Implement the ingest worker: GET boards-api.greenhouse.io/v1/boards/{token}/jobs?content=true, no auth required
- Validate each raw response against a Zod schema before writing it to raw_signal
- Normalize: parse into the hiring_signal shape, resolving or creating the company row on name and domain match

EXIT CRITERIA

Real postings from real companies are landing as clean hiring_signal rows on a working schedule.

DAY 5 | Signal Source: Lever

- Apply the same fetch pattern to Lever's public job board API (api.lever.co, no auth required)
- Validate against the same Zod schema pattern and write to raw_signal
- Normalize into the same hiring_signal shape used by Greenhouse, proving the pattern is source-agnostic

EXIT CRITERIA

raw_signal and hiring_signal are populated from two independent sources on working schedules.

DAY 6 | Identity Handling, Cross-Source Dedup, and Early Filter

- Dedup within the freshness window, checked across both sources, so the same job posted on both Greenhouse and Lever collapses to one hiring_signal row rather than two
- Apply the early structural checks available at this point in the pipeline: freshness ceiling and record completeness. Contact-dependent rules such as reachability move to the day 10 quality gate, once a contact exists to check
- Log filtered and deduped counts per run, split by source, for visibility

EXIT CRITERIA

hiring_signal rows are deduped both within and across the two sources, and pass a basic sanity filter before enrichment.

DAY 7 | Margin Day: Foundation and Signal Pipeline

MARGIN DAY — NO NEW FEATURES SCHEDULED

Catch up on anything from days 1 through 6 that didn't land cleanly. Re-test the schema, RLS policies, queue setup, and the shared lead type under normal use, and audit a sample of hiring_signal rows from both sources for accuracy before enrichment touches them.

WEEK 2 — Enrichment, Emission, and a Thin Interface

DAY 8 | Basic Enrichment and Contact Selection

- Wire Seamless.AI into the enrich worker, queried by company and likely title
- Write hits to contact with confidence_score and source
- Basic contact selection: star the single highest-confidence contact returned. The full weighted algorithm (functional match, geographic match, seniority calibration) is explicitly deferred to the next phase
- Set a concurrency limit on this queue, since every call has a real cost

EXIT CRITERIA

Real contacts are attached to hiring_signal rows, with one contact starred per candidate lead.

DAY 9 | Basic Per-Tenant Configuration

- Add a minimal read path for the configuration table: target industry, target geography, and target company size band per tenant
- Wire the scoring stage to read the requesting tenant's actual configuration row instead of a hardcoded rule
- Seed a real configuration row for the test tenant reflecting one genuine target market

EXIT CRITERIA

The fit check in scoring evaluates against a real per-tenant configuration row pulled from the database, not a hardcoded constant.

DAY 10 | Basic Scoring and Emission

- Compute the freshness band (fresh, recent, ageing, stale) per signal
- Apply the fit check against the requesting tenant's configuration row, wired in on day 9
- Apply the reachability quality gate: a contact with no validated dialable number is dropped here, not delivered
- Emit: write the finished lead (global) and a lead_assignment (tenant-scoped) for the test tenant, using a hardcoded placeholder in place of generated intelligence

EXIT CRITERIA

A real lead_assignment row exists in Supabase, correctly shaped against the frozen contract, correctly scored against real tenant configuration, and correctly isolated to the test tenant.

DAY 11 | End-to-End Pipeline Test

- Run the full chain, ingest through emit, against real data with no manual steps
- Confirm a single real posting produces one correctly-shaped, correctly-isolated lead
- Fix whatever breaks under a real full run rather than isolated stage tests

EXIT CRITERIA

The sentence 'a real posting went through every stage and landed correctly in one tenant's queue' is true, repeatably, not as a one-off.

DAY 12 | Interface Foundation and Queue View

- Scaffold the React and Vite frontend with Tailwind, dropping in the Full Desk Leads brand tokens: green #163E33 primary, ivory #F8F8F5 background, charcoal #1E2221 body text
- Wire authentication through Supabase Auth
- Build the queue view: a list of the logged-in tenant's lead_assignment rows, served through a thin Fastify read endpoint

EXIT CRITERIA

A logged-in test user sees their own tenant's leads in a list, and only their own tenant's leads.

DAY 13 | Lead Card and Basic Outcome Capture

- Build the lead card component covering the Layer 1 fields only: company name, contact name and title, phone, freshness tag, and the placeholder why-now text. Layer 2 expansion is deferred
- Implement one outcome action that writes an interaction_event row against the correct lead_assignment

EXIT CRITERIA

A user can open a real lead's card from the queue and log one outcome, correctly stored against that assignment.

DAY 14 | Margin Day: Stabilization and Wrap-up

MARGIN DAY — NO NEW FEATURES SCHEDULED

Add structured logging (pino) across the workers. Fix whatever is still shaky from the preceding 13 days. Write the repo README covering what's built, how to run it, and exactly what phase two covers. No new feature work starts today.

Explicitly Deferred to Phase Two

- Additional signal sources: Ashby, JobsPipe
- Full identity resolution: entity mismatch detection, quarantine, reversible merge log
- The full three-tier filter: universal exclusion lists, tenant exclusions
- Enrichment fallback (Apollo), role research, and the role family knowledge base
- The full weighted contact selection algorithm, ranked alternates, and auto-promotion
- LLM intelligence generation with fact-validation guardrails, replacing the placeholder pitch
- The full four-score system (fit, freshness, confidence, priority) and all eight quality gates
- Lead card Layer 2, the follow-up scheduler, history timeline, and opportunities
- The configuration editing screen itself, admin dashboard, and the internal operations console
- Mobile responsive behaviour and the audit log